Radar passif multicanal sur KrakenSDR : détection OS-CFAR et direction d'arrivée par la phase sur un réseau de quatre antennes

Yanis Djelali, ENSEIRB-MATMECA, filière télécommunications

Rapport technique, octobre 2026. Code : fork de `krakenrf/krakensdr_pr`.

Résumé

Ce rapport décrit un radar passif bistatique exploitant un émetteur DVB-T et un récepteur cohérent à cinq voies KrakenSDR : une voie de référence pointée vers l'émetteur, quatre voies de surveillance en réseau linéaire. Pour chaque intervalle de traitement de 2^{20} échantillons (0,44 s), la chaîne supprime le trajet direct par un filtre de Wiener, calcule une carte distance/Doppler par antenne, détecte les échos par un OS-CFAR bidimensionnel sur la somme incohérente des cartes, puis estime la direction d'arrivée de chaque écho à partir des phases relatives entre antennes (Bartlett, Capon ou MUSIC sur une matrice de covariance accumulée sur plusieurs intervalles). Le travail d'origine (extension multicanal, CFAR, Capon) a été audité et corrigé : seuil CFAR faux de 11 dB, somme cohérente aveugle à $\pm 30°$, estimateur Capon dégénéré sur un seul instantané. Sur une scène synthétique, trois cibles à 50 à 56 dB sous le trajet direct sont retrouvées à la cellule exacte et à $0{,}1°$ près. Le logiciel a été déployé sur un Raspberry Pi 5 et un KrakenSDR réels : les cinq voies sont synchronisées et calibrées, et la carte s'affiche en temps réel. L'observation d'une cible réelle reste à faire.

1 Introduction

Un radar passif n'émet pas : il utilise un émetteur d'opportunité, ici un multiplex de télévision numérique terrestre (DVB-T, 569 MHz, 8 MHz de bande), et mesure les échos qu'il produit sur des cibles mobiles. Le signal DVB-T est bien adapté car il est continu, large bande (donc fin en distance) et sans structure périodique forte dans la bande utile.

Le KrakenSDR est un récepteur à cinq tuners RTL-SDR partageant la même horloge, avec une source de bruit interne qui permet de calibrer la phase entre voies. Le logiciel de radar passif publié par le constructeur n'utilise que deux voies (référence et surveillance) et ne donne ni détection automatique ni direction. Le projet consiste à exploiter les quatre voies restantes comme un réseau d'antennes afin d'obtenir, pour chaque écho, un détour, un Doppler et un angle d'arrivée.

Le présent document résume la méthode, les erreurs trouvées dans la première version et les résultats obtenus en simulation et sur matériel. Les démonstrations détaillées figurent dans deux documents du dépôt (`docs/`).

2 Méthode

2.1 Modèle du signal

L'émetteur rayonne $s(t)$. La voie de référence reçoit $s(t)$ presque pur. Une voie de surveillance m reçoit le trajet direct, des réflexions fixes (clutter) et l'écho d'une cible :

$$x_m(t) = \sum_k c_{k,m}\, s(t-\tau_k) + A\, a_m(\theta)\, s(t-\tau)\, e^{j2\pi f_D t} + n_m(t), \tag{1}$$

où τ est le retard de l'écho par rapport au trajet direct, f_D le Doppler bistatique et $a_m(\theta)$ le facteur de phase lié à la géométrie du réseau. Pour un réseau linéaire de pas d :

$$a_m(\theta) = \exp\!\left(j\, 2\pi\, m\, \frac{d}{\lambda} \sin\theta\right), \qquad m = 0, \ldots, M-1. \tag{2}$$

Avec $d = \lambda/2$ (26,3 cm à 569 MHz), le déphasage entre antennes voisines vaut $\pi \sin\theta$, univoque sur $[-90°, 90°]$. Le détour $R_b = c\tau$ se lit en cellules de $c/f_s = 125$ m ($f_s = 2{,}4$ MHz). Le Doppler se lit en cases de $f_s/2N = 1{,}14$ Hz, soit 0,527 m/s de vitesse bistatique par hertz.

2.2 Suppression du clutter

Le trajet direct et le clutter dominent l'écho de 50 à 80 dB. On les prédit à partir de la référence retardée de 0 à $K-1$ cellules ($K = 128$) et on soustrait la prédiction (filtre de Wiener, variante SMI-MRE du code amont) :

$$\hat{\mathbf{w}} = \mathbf{R}^{-1}\mathbf{r}, \quad \mathbf{R} = \mathbf{S}^{\mathsf{H}}\mathbf{S}, \quad \mathbf{r} = \mathbf{S}^{\mathsf{H}}\mathbf{x}_m, \tag{3}$$

où $\mathbf{S}$ empile les versions retardées de la référence. $\mathbf{R}$ est Toeplitz : sa première colonne suffit, calculée par corrélation FFT. Un écho de Doppler f_D tourne en phase pendant l'intervalle T et n'est pas prédit par une combinaison fixe de retards : il survit, atténué de $\mathrm{sinc}(f_D T)$, ce qui laisse un fantôme à Doppler nul à sa distance. Ce fantôme est exclu de la détection par un masque de deux lignes autour du Doppler nul.

2.3 Carte distance/Doppler

La fonction d'ambiguïté croisée est calculée par la méthode des blocs : l'intervalle de N échantillons est découpé en $P = N/K$ blocs, chaque bloc est corrélé avec la référence par FFT (axe distance, K cellules), puis une FFT sur l'indice de bloc donne l'axe Doppler. Le gain d'intégration vaut $10\log_{10} N = 60$ dB, et la méthode impose $|f_D| < f_s/2K = 9{,}4$ kHz, largement suffisant. Une carte complexe $\chi_m(R, f_D)$ est obtenue par antenne.

2.4 Détection

La statistique de détection est la somme incohérente

$$Z(R, f_D) = \left(\sum_{m=1}^{M} |\chi_m(R, f_D)|^2 \right)^{1/2}, \qquad (4)$$

qui ne dépend pas de l'angle d'arrivée. La somme cohérente $|\sum_m \chi_m|$ utilisée dans la version d'origine est le diagramme d'un faisceau pointé à $0°$: pour $M = 4$ et $d = \lambda/2$, il possède des zéros exacts à $\pm 30°$ et $\pm 90°$.

Le détecteur est un OS-CFAR 2D : pour chaque cellule, le bruit est estimé par le percentile $p = 0{,}75$ des cellules d'apprentissage (16 cellules de chaque côté, 3 de garde), et la cellule est déclarée si $Z > \alpha\,\hat{\sigma}$. Pour une amplitude de Rayleigh, le quantile p vaut $\sigma\sqrt{-2\ln(1-p)}$ et $P(Z > \eta) = e^{-\eta^2/2\sigma^2}$, d'où

$$\alpha = \frac{\sqrt{\ln(1/P_{fa})}}{\sqrt{-\ln(1-p)}} \quad (\alpha = 2{,}88,\ 9{,}2 \text{ dB pour } P_{fa} = 10^{-5}). \qquad (5)$$

Une seule détection est conservée par maximum local 3×3.

2.5 Direction d'arrivée

Pour une cellule détectée, les M valeurs complexes $\mathbf{x} = [\chi_1, \ldots, \chi_M]^{\mathsf{T}}$ forment un instantané qui, en l'absence de bruit, vaut $A\,\mathbf{a}(\theta)$. L'estimation consiste à tester chaque angle : on corrige les phases prédites et on mesure la cohérence, $|\mathbf{a}(\theta)^{\mathsf{H}}\mathbf{x}|^2$, maximale au bon angle. Pour aller au-delà, on accumule L instantanés (les L_c derniers intervalles et les 8 cellules voisines) dans $\mathbf{R} = \frac{1}{L}\sum_\ell \mathbf{x}_\ell \mathbf{x}_\ell^{\mathsf{H}}$ et on calcule

$$P_{\mathrm{B}} = \mathbf{a}^{\mathsf{H}}\mathbf{R}\,\mathbf{a}, \qquad P_{\mathrm{C}} = \frac{1}{\mathbf{a}^{\mathsf{H}}\mathbf{R}^{-1}\mathbf{a}}, \qquad P_{\mathrm{M}} = \frac{1}{\mathbf{a}^{\mathsf{H}}\mathbf{E}_n\mathbf{E}_n^{\mathsf{H}}\mathbf{a}}, \qquad (6)$$

avec $\mathbf{E}_n$ les vecteurs propres de bruit de $\mathbf{R}$. Avec un seul instantané, $\mathbf{R} = \mathbf{x}\mathbf{x}^{\mathsf{H}}$ est de rang 1 ; après chargement diagonal, l'identité de Sherman-Morrison montre que P_{C} est une fonction croissante de P_{B} : Capon n'apporte alors rien, ce qui était le cas de la version d'origine. Le code retombe automatiquement sur Bartlett si $L < M$.

2.6 Mise en œuvre

La chaîne est une fonction pure `process_cpi(iq, fs, settings)` appelée par le thread de traitement et par un simulateur hors ligne. Les boucles (Wiener, CFAR, maxima locaux) sont compilées par Numba, les FFT par pyFFTW. Le CFAR traite les lignes en parallèle, obtient le quantile par sélection (`np.partition`, $O(n)$) plutôt que par tri, et ne teste que les maxima locaux. L'interface web (Dash 1.20, pousser par websocket) affiche la carte en km et Hz, normalisée par la médiane du bruit, avec une croix et une étiquette (détour, Doppler, angle, SNR) par détection.

3 Résultats

3.1 Scène synthétique

Référence gaussienne complexe, trajet direct et deux réflexions fixes à -10 et -20 dB, trois cibles à -50, -50 et -56 dB sous le trajet direct, aux angles $+30°$, $0°$, $-45°$, $N = 2^{18}$, quatre antennes à $\lambda/2$. Le tableau 1 donne les résultats de la chaîne corrigée (Bartlett, un intervalle) ; la figure 1 montre la carte. Aucune fausse alarme n'apparaît sur les 57 000 cellules.

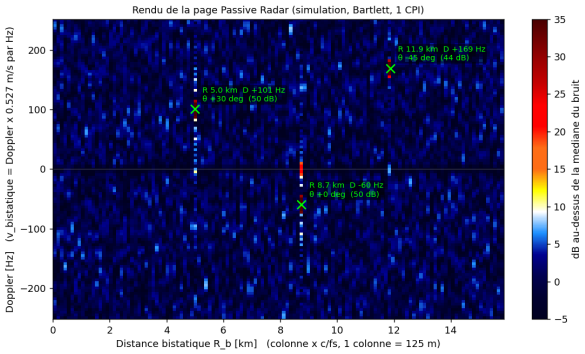


Fig. 1 : Carte distance/Doppler simulée, somme incohérente des quatre antennes, en dB au-dessus de la médiane du bruit. Croix : détections OS-CFAR, étiquettes : détour, Doppler, angle Bartlett, SNR.

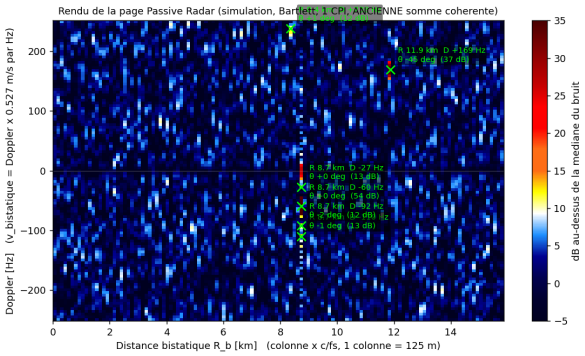


Fig. 2 : Même scène avec la somme cohérente de la version d'origine : cible à $+30°$ invisible, fausses détections sur les lobes secondaires.

Avec la statistique de la version d'origine (figure 2), la cible à $+30°$ disparaît, la cible à $-45°$ perd 7 dB et la cible à $0°$, amplifiée de 6 dB, fait passer ses lobes secondaires au-dessus du seuil : trois fausses détections.

Après six intervalles (36 instantanés par cellule), MUSIC donne des pics de moins de $2°$ de large aux trois angles (figure 3).

3.2 Seuil CFAR

La version d'origine calculait $\alpha = N_t\,(P_{fa}^{-1/N_t} - 1)$, formule du CA-CFAR sur des puissances, appliquée à un percentile d'amplitudes. Avec $N_t = 1472$ cellules d'apprentissage et $P_{fa} = 10^{-4}$, $\alpha = 9{,}24$, soit un seuil à $10{,}9\,\sigma$ (20,7 dB) au lieu de $3{,}04\,\sigma$ (9,6 dB) ; la probabilité de fausse alarme réelle était de l'ordre de 10^{-49} et la sensibilité perdait 11 dB. L'équation (5) corrige ce point ; P_{fa} est maintenant un réglage de l'interface.

3.3 Matériel

Le logiciel a été installé sur l'image officielle KrakenSDR pour Raspberry Pi 5 (23 septembre 2026),

	Attendu	Obtenu	SNR
Cible 1	5,00 km, $+100$ Hz, $+30°$	5,00 km, $+100{,}7$ Hz, $+30{,}0°$	49,7 dB
Cible 2	8,74 km, -60 Hz, $0°$	8,74 km, $-59{,}5$ Hz, $0{,}0°$	49,8 dB
Cible 3	11,87 km, $+170$ Hz, $-45°$	11,87 km, $+169{,}4$ Hz, $-45{,}0°$	43,8 dB

Tab. 1 : Détections sur la scène synthétique (chaîne corrigée). Les écarts en Hz sont inférieurs à la demi-case Doppler ($4{,}6$ Hz à $N = 2^{18}$).

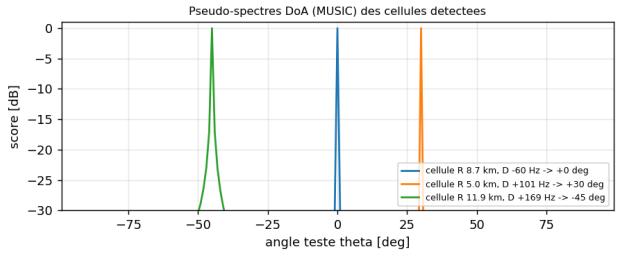


Fig. 3 : Pseudo-spectres MUSIC des trois cellules détectées après six intervalles.

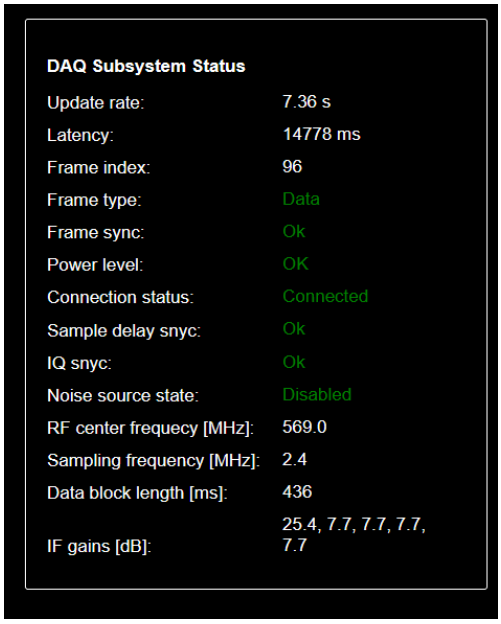


Fig. 4 : État du DAQ sur le Raspberry Pi 5 : 5 voies, synchronisation IQ acquise, gains 25,4 / 7,7 dB.

antennes posées en intérieur pour tester la chaîne. Les cinq tuners sont reconnus, le DAQ fonctionne en configuration 5 voies, 2^{20} échantillons, 2,4 MHz à 569 MHz ; la calibration de retard et de phase entre voies est acquise (`IQ sync Ok`, figure 4), les cinq gains sont appliqués et la carte se rafraîchit dans le navigateur. Le temps de traitement mesuré par intervalle était de 7,36 s avant l'optimisation du CFAR (le profil du simulateur attribue l'essentiel au tri des fenêtres d'apprentissage : 3,2 s en numpy pur sur PC, 0,28 s après réécriture). Le temps par intervalle sur Pi 5 avec la version optimisée n'a pas encore été mesuré.

4 Discussion

Les trois corrections de conception (somme incohérente, seuil de Rayleigh, covariance multi-instantanés) découlent chacune d'une propriété simple : le diagramme d'un réseau sommé en phase a des zéros ; un percentile d'amplitude n'est pas une moyenne de puissance ; une matrice de rang 1 ne peut pas séparer un signal de son bruit. La simulation reproduit le défaut de la version d'origine (cible à $30°$ perdue), ce qui en fait un test de non-régression.

Les limites sont celles d'un réseau linéaire (ambiguïté avant/arrière, précision dégradée vers $\pm 90°$) et de l'absence de validation sur cible réelle : la convention de signe de l'angle et les déphasages de câbles (`doa_phase_offsets_deg`, prévu mais à zéro) doivent être mesurés sur un avion suivi en ADS-B. La position au sol (intersection de l'ellipse de détour et de l'azimut) et le pistage ne sont pas implémentés. Enfin la contrainte temps réel sur Raspberry Pi 5 avec quatre canaux reste à confirmer ; les leviers sont la taille de la fenêtre CFAR, le détour maximal, le nombre de canaux et la longueur de l'intervalle.

Contributions

Travail d'origine (extension multicanal, OS-CFAR 2D, Capon) : l'auteur. Audit, réécriture de la chaîne, interface, simulateur, tests et documentation : réalisés avec l'assistance d'un modèle de langage. Installation, mise en service du matériel et validation sur Raspberry Pi 5 et KrakenSDR : l'auteur.

Références

[1] KrakenRF, *krakensdr_pr*, *heimdall_daq_fw*, *krakensdr_docs* (wiki « Passive Radar »), `https://github.com/krakenrf`.

[2] T. Pető, *pyAPRiL, Python Advanced Passive Radar Library*, `https://github.com/petotamas/pyAPRiL` (méthode des blocs et filtre SMI-MRE repris par le code amont).

[3] H. Rohling, « Radar CFAR thresholding in clutter and multiple target situations », *IEEE Trans. AES*, 1983 (OS-CFAR).

[4] R. Schmidt, « Multiple emitter location and signal parameter estimation », *IEEE Trans. AP*, 1986 (MUSIC).

[5] J. Capon, « High-resolution frequency-wavenumber spectrum analysis », *Proc. IEEE*, 1969.

[6] H. L. Van Trees, *Optimum Array Processing*, Wiley, 2002 (Bartlett, Capon, modèle de réseau linéaire).